

JavaScript

From Scratch

Complete Beginner's Guide

Variables & Data Types	Functions & Scope	Arrays & Objects
Control Flow	ES6+ Features	DOM & Events
Loops	Promises & Async	Error Handling

A comprehensive, hands-on guide covering all essential JavaScript concepts — with clear explanations, code examples, and key takeaways.

■ Introduction to JavaScript

JavaScript (JS) is a lightweight, interpreted, multi-paradigm programming language. It was created in 1995 by Brendan Eich at Netscape in just 10 days. Today it powers interactive websites, server-side apps (Node.js), mobile apps, and even desktop software.

■ Why Learn JavaScript?

- Only language natively supported by all web browsers
- Runs on both client (browser) and server (Node.js)
- Huge ecosystem: npm has 2 million+ packages
- Foundation for frameworks like React, Vue, Angular, Next.js
- High demand in the job market worldwide

■ How to Run JavaScript

```
// 1. Browser Console – press F12 → Console tab
console.log("Hello, World!");

// 2. HTML file – embed in <script> tag
<script>
alert("Welcome to JS!");
</script>

// 3. Node.js – run from terminal
// $ node myfile.js
```

■ Use `console.log()` constantly while learning — it's your best debugging tool.

■ Variables & Data Types

■ Declaring Variables

JavaScript provides three keywords to declare variables: **var**, **let**, and **const**. Modern JS (ES6+) strongly prefers **let** and **const**.

```
var age = 25; // old-school, function-scoped, avoid in modern code
let name = 'Sai'; // block-scoped, reassignable
const PI = 3.14159; // block-scoped, cannot be reassigned

name = 'Ravi'; // ■ allowed – let is mutable
PI = 3; // ■ TypeError – const is immutable
```

Keyword	Behaviour
var	Function-scoped, hoisted, avoid
let	Block-scoped, reassignable ■
const	Block-scoped, immutable reference ■

■ Primitive Data Types

```
let num = 42; // Number
let pi = 3.14; // Number (float)
let str = 'Hello'; // String
let isReady = true; // Boolean
let nothing = null; // Null (intentional empty)
let undef; // Undefined (declared, not assigned)
let id = Symbol('id'); // Symbol (unique identifier)
let big = 9007199254740993n; // BigInt
```

■ typeof Operator

```
typeof 42 // "number"
typeof "hello" // "string"
typeof true // "boolean"
typeof undefined // "undefined"
typeof null // "object" ← known JS quirk!
typeof {} // "object"
typeof [] // "object"
typeof function(){} // "function"
```

- Use `Array.isArray([])` to check for arrays, not `typeof`.

■ Type Coercion & Conversion

```
// Implicit coercion (JS does it automatically)
"5" + 2 // "52" – number converted to string
"5" - 2 // 3 – string converted to number
true + 1 // 2 – boolean to number

// Explicit conversion
Number("42") // 42
String(100) // "100"
Boolean(0) // false
parseInt("10px") // 10
parseFloat("3.7em") // 3.7
```

■ Operators

■ Arithmetic Operators

```
let a = 10, b = 3;  
a + b // 13 addition  
a - b // 7 subtraction  
a * b // 30 multiplication  
a / b // 3.33 division  
a % b // 1 modulo (remainder)  
a ** b // 1000 exponentiation (ES7)  
++a // 11 pre-increment  
b-- // 3 post-decrement
```

■ Comparison Operators

```
5 == '5' // true – loose equality (type coercion)  
5 === '5' // false – strict equality (no coercion) ■ prefer this  
5 != '5' // false  
5 !== '5' // true ■ prefer this  
10 > 5 // true  
10 >= 10 // true  
3 < 5 // true
```

■ Always use `===` and `!==` to avoid unexpected type coercion bugs.

■ Logical & Assignment Operators

```
// Logical
true && false // false (AND)
true || false // true (OR)
!true // false (NOT)

// Assignment shorthand
let x = 10;
x += 5; // x = 15
x -= 3; // x = 12
x *= 2; // x = 24
x /= 4; // x = 6
x **= 2; // x = 36

// Nullish coalescing (ES2020)
let val = null ?? "default"; // "default"

// Optional chaining (ES2020)
let user = null;
let city = user?.address?.city; // undefined (no error)
```

■ Control Flow

■ if / else if / else

```
let score = 75;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 75) {
  console.log("Grade: B");
} else if (score >= 60) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
// Output: Grade: B
```

■ Ternary Operator

```
// condition ? valueIfTrue : valueIfFalse
let age = 20;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status); // "Adult"
```

■ switch Statement

```
let day = "Monday";

switch (day) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  case "Monday":
    console.log("Back to work.");
    break;
  default:
    console.log("Weekday");
}
```

■ Always add break in switch cases to prevent fall-through — unless intentional.

■ Loops

■ for Loop

```
// Syntax: for(initializer; condition; update)
for (let i = 0; i < 5; i++) {
  console.log(i); // 0 1 2 3 4
}
```

■ while & do...while

```
// while – checks condition BEFORE executing
let count = 0;
while (count < 3) {
  console.log(count); // 0 1 2
  count++;
}

// do...while – executes AT LEAST ONCE
let n = 10;
do {
  console.log(n); // 10 (runs once even though n >= 3)
  n++;
} while (n < 3);
```

■ for...of & for...in

```
// for...of – iterates over VALUES (arrays, strings, Maps)
let fruits = ['apple', 'banana', 'mango'];
for (let fruit of fruits) {
  console.log(fruit); // apple banana mango
}

// for...in – iterates over KEYS of an object
let person = { name: 'Sai', age: 21, city: 'Hyderabad' };
for (let key in person) {
  console.log(key, ': ', person[key]);
}
// name : Sai | age : 21 | city : Hyderabad
```

■ Loop Control: break & continue

```
for (let i = 0; i < 10; i++) {  
  if (i === 3) continue; // skip 3  
  if (i === 7) break; // stop at 7  
  console.log(i); // 0 1 2 4 5 6  
}
```

■ Functions

■ Function Declaration

```
function greet(name) {  
  return "Hello, " + name + "!";  
}  
  
console.log(greet("Sai")); // Hello, Sai!  
  
// Declarations are HOISTED – can be called before they appear in code
```

■ Function Expression

```
const square = function(n) {  
  return n * n;  
};  
  
console.log(square(5)); // 25  
// Not hoisted – must be defined before use
```

■ Arrow Functions (ES6)

```
// Full form  
const add = (a, b) => {  
  return a + b;  
};  
  
// Concise form (single expression – implicit return)  
const multiply = (a, b) => a * b;  
  
// Single parameter – no parentheses needed  
const double = n => n * 2;  
  
console.log(add(3, 4)); // 7  
console.log(multiply(3, 4)); // 12  
console.log(double(6)); // 12
```

■ Default & Rest Parameters

```
// Default parameter
function greet(name = "World") {
  return `Hello, ${name}!`;
}
greet(); // "Hello, World!"
greet("Ravi"); // "Hello, Ravi!"

// Rest parameters – collect extra args into array
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4, 5); // 15
```

■ Scope & Closures

```
// Closures – inner function remembers outer scope
function makeCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const counter = makeCounter();
counter(); // 1
counter(); // 2
counter(); // 3
```

■ A closure gives an inner function access to variables from its outer function, even after the outer function has finished executing.

■ Arrays

■ Creating & Accessing Arrays

```
let colors = ["red", "green", "blue"];
let nums = [10, 20, 30, 40, 50];

colors[0] // 'red' (zero-indexed)
colors[2] // 'blue'
colors.length // 3
colors[colors.length - 1] // 'blue' (last element)
```

■ Common Array Methods

```
let arr = [1, 2, 3];

arr.push(4); // [1,2,3,4] – add to end
arr.pop(); // [1,2,3] – remove from end
arr.unshift(0); // [0,1,2,3] – add to start
arr.shift(); // [1,2,3] – remove from start
arr.indexOf(2); // 1 – find index
arr.includes(3); // true
arr.reverse(); // [3,2,1]
arr.join(', '); // '3, 2, 1'
arr.slice(0, 2); // [3, 2] – non-destructive
arr.splice(1, 1); // removes 1 element at index 1
```

■ Higher-Order Array Methods

```
let numbers = [1, 2, 3, 4, 5];

// map – transform each element → new array
let doubled = numbers.map(n => n * 2);
// [2, 4, 6, 8, 10]

// filter – keep elements that pass test → new array
let evens = numbers.filter(n => n % 2 === 0);
// [2, 4]

// reduce – accumulate to single value
let total = numbers.reduce((acc, n) => acc + n, 0);
// 15

// find – first element that passes test
let found = numbers.find(n => n > 3);
// 4

// some – true if ANY element passes
numbers.some(n => n > 4); // true

// every – true if ALL elements pass
numbers.every(n => n > 0); // true
```

■ ■ Objects

■ Creating Objects

```
// Object literal
let student = {
  name: 'Sai',
  age: 21,
  college: 'RVIT',
  grades: [90, 85, 92],
  address: {
    city: 'Hyderabad',
    state: 'Telangana'
  }
};

// Accessing properties
student.name // 'Sai' – dot notation
student['college'] // 'RVIT' – bracket notation
student.address.city // 'Hyderabad' – nested
```

■ Modifying Objects

```
student.email = 'sai@example.com'; // add new property
student.age = 22; // update property
delete student.grades; // remove property

// Check if property exists
'name' in student // true
student.hasOwnProperty('email') // true
```

■ Object Methods

```
let car = {
  brand: 'Toyota',
  speed: 0,
  accelerate(amount) {
    this.speed += amount;
    console.log(`Speed: ${this.speed} km/h`);
  }
};

car.accelerate(60); // Speed: 60 km/h
car.accelerate(40); // Speed: 100 km/h
```

■ Object Utility Methods

```
let obj = { a: 1, b: 2, c: 3 };

Object.keys(obj); // ['a', 'b', 'c']
Object.values(obj); // [1, 2, 3]
Object.entries(obj); // [['a',1], ['b',2], ['c',3]]

// Shallow clone
let copy = Object.assign({}, obj);
let copy2 = { ...obj }; // spread operator clone

// Merge objects
let merged = { ...obj, d: 4 }; // { a:1, b:2, c:3, d:4 }
```

■ ES6+ Modern JavaScript

■ Template Literals

```
let name = 'Sai';
let score = 95;

// Old way
"Hello " + name + ", your score is " + score;

// Template literal (backtick)
`Hello ${name}, your score is ${score}!`

// Multi-line strings
let poem = `Roses are red,
Violets are blue`;

// Expressions inside ${}
`2 + 2 = ${2 + 2}` // "2 + 2 = 4"
```

■ Destructuring

```
// Array destructuring
let [a, b, c] = [10, 20, 30];
console.log(a, b, c); // 10 20 30

// Skip elements
let [first, , third] = [1, 2, 3]; // 1 3

// Object destructuring
let { name, age, city = 'Unknown' } = { name: 'Sai', age: 21 };
// name='Sai' age=21 city='Unknown' (default)

// Rename while destructuring
let { name: fullName } = { name: 'Sai Kumar' };
console.log(fullName); // 'Sai Kumar'
```

■ Spread & Rest Operators

```
// Spread – expand an iterable
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let combined = [...arr1, ...arr2]; // [1,2,3,4,5,6]

// Pass array as individual args
Math.max(...arr1); // 3

// Object spread
let defaults = { theme: 'dark', lang: 'en' };
let config = { ...defaults, lang: 'hi', debug: true };
// { theme:'dark', lang:'hi', debug:true }

// Rest – collect remaining into array
let [head, ...tail] = [1, 2, 3, 4];
// head=1 tail=[2,3,4]
```

■ Short-circuit & Nullish Coalescing

```
// || – fallback if left is FALSY (0, '', false, null, undefined)
let username = "" || "Guest"; // "Guest"

// ?? – fallback ONLY if left is null or undefined
let username2 = "" ?? "Guest"; // "" (empty string is kept)

// && – short-circuit evaluation
let user = { name: 'Sai' };
user && console.log(user.name); // prints 'Sai'

// Optional chaining
let city = user?.address?.city; // undefined, no error
```

■ Strings

■ Common String Methods

```
let str = " Hello, World! ";

str.length // 18
str.trim() // 'Hello, World!'
str.toUpperCase() // ' HELLO, WORLD! '
str.toLowerCase() // ' hello, world! '
str.includes('World') // true
str.startsWith(' Hello') // true
str.endsWith('! ') // true
str.indexOf('World') // 8
str.slice(2, 7) // 'Hello'
str.replace("World", "JS") // " Hello, JS! "
str.split(", ") // [" Hello", "World! "]
"ha".repeat(3) // "hahaha"
"5".padStart(3, "0") // "005"
```

■ DOM Manipulation

The Document Object Model (DOM) is a tree of HTML elements that JavaScript can read and modify. Every tag becomes a node you can select, update, or delete.

■ Selecting Elements

```
// Single element
document.getElementById('myId');
document.querySelector('.myClass'); // first match
document.querySelector('h1');

// Multiple elements → NodeList
document.querySelectorAll('p');
document.getElementsByClassName('btn');
document.getElementsByTagName('div');
```

■ Modifying Elements

```
let el = document.querySelector('#title');

// Content
el.textContent = "New Title"; // plain text
el.innerHTML = "<b>Bold</b> text"; // HTML string

// Attributes
el.setAttribute("class", "highlight");
el.getAttribute("id"); // "title"
el.removeAttribute('disabled');

// CSS Styles
el.style.color = "red";
el.style.fontSize = "24px";
el.style.display = "none"; // hide element

// Classes
el.classList.add('active');
el.classList.remove('hidden');
el.classList.toggle('selected');
el.classList.contains('active'); // true
```

■ Creating & Removing Elements

```
// Create
let newDiv = document.createElement("div");
newDiv.textContent = "I am new!";
document.body.appendChild(newDiv);

// Insert before another element
let ref = document.querySelector("#ref");
ref.parentNode.insertBefore(newDiv, ref);

// Modern: insertAdjacentHTML
ref.insertAdjacentHTML("afterend", "<p>After ref</p>");

// Remove
newDiv.remove();
```

■ Events

■ Adding Event Listeners

```
let btn = document.querySelector("#myBtn");

// Preferred: addEventListener
btn.addEventListener('click', function(event) {
  console.log("Button clicked!", event);
});

// Arrow function
btn.addEventListener('mouseover', e => {
  e.target.style.background = "yellow";
});

// Remove listener
function handleClick() { console.log('clicked'); }
btn.addEventListener('click', handleClick);
btn.removeEventListener('click', handleClick);
```

■ Common Events

Event	When it fires
click	Mouse click
dblclick	Double click
mouseover	Hover over element
keydown	Key pressed
keyup	Key released
submit	Form submitted
input	Input value changes
change	Input value committed
load	Page fully loaded
DOMContentLoaded	DOM ready (no images)

■ Event Object & Bubbling

```
document.querySelector('form').addEventListener('submit', e => {  
  e.preventDefault(); // stop form from reloading page  
  console.log(e.target); // the form element  
});  
  
// Event bubbling – events propagate up to parent  
// Use stopPropagation() to prevent bubbling  
child.addEventListener('click', e => {  
  e.stopPropagation();  
  console.log('child clicked only');  
});
```

■ Promises & Async/Await

JavaScript is single-threaded. Asynchronous operations (network requests, timers, file I/O) are handled without blocking the thread using Promises and async/await.

■ Callbacks (old way)

```
setTimeout(function() {
  console.log("Runs after 2 seconds");
}, 2000);

// Callback hell – deeply nested, hard to read
getData(function(a) {
  getMoreData(a, function(b) {
    getEvenMore(b, function(c) {
      // ...nightmare
    });
  });
});
```

■ Promises

```
// Create a Promise
let fetchData = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Data fetched!");
  } else {
    reject("Error fetching data");
  }
});

// Consume a Promise
fetchData
  .then(data => console.log(data)) // success
  .catch(err => console.error(err)) // failure
  .finally(() => console.log('Done')); // always runs
```

■ Async / Await (ES2017)

```
// async function always returns a Promise
async function loadUser(id) {
  try {
    let res = await fetch(`/api/users/${id}`);
    let user = await res.json();
    console.log(user.name);
  } catch (err) {
    console.error('Failed:', err);
  }
}

loadUser(1);

// Promise.all – run multiple promises in parallel
let [users, posts] = await Promise.all([
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
]);
```

■ Use `async/await` for cleaner, readable async code. Always wrap in `try/catch`!

■ Error Handling

■ try / catch / finally

```
try {
  let result = riskyOperation();
  console.log(result);
} catch (error) {
  console.error('Error name:', error.name);
  console.error('Error message:', error.message);
} finally {
  console.log('This always runs – cleanup here');
}
```

■ Throwing Custom Errors

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed");
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (e) {
  console.error(e.message); // Division by zero is not allowed
}
```

■ Error Types

Type	When it occurs
Error	Generic error base class
TypeError	Wrong type used
ReferenceError	Variable not defined
SyntaxError	Invalid JS syntax
RangeError	Value out of allowed range
URIError	Invalid URI function usage

■ ■ Classes (OOP)

■ Defining Classes

```
class Animal {  
  // constructor – called when object is created  
  constructor(name, sound) {  
    this.name = name;  
    this.sound = sound;  
  }  
  
  // method  
  speak() {  
    console.log(`${this.name} says ${this.sound}!`);  
  }  
  
  // static method – called on the class, not instance  
  static describe() {  
    console.log("Animals are living beings.");  
  }  
}  
  
let dog = new Animal('Dog', 'Woof');  
dog.speak(); // Dog says Woof!  
Animal.describe(); // Animals are living beings.
```

■ Inheritance

```
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name, "Woof"); // call parent constructor  
    this.breed = breed;  
  }  
  
  fetch() {  
    console.log(`${this.name} fetches the ball!`);  
  }  
}  
  
let lab = new Dog("Buddy", "Labrador");  
lab.speak(); // Buddy says Woof! (inherited)  
lab.fetch(); // Buddy fetches the ball!
```

■ Modules (ES6)

Modules let you split code into separate files and import/export only what you need, keeping your codebase organized and maintainable.

■ Exporting

```
// math.js
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

// Default export (one per file)
export default class Calculator { /* ... */ }
```

■ Importing

```
// main.js
import { PI, add, multiply } from './math.js';

console.log(PI); // 3.14159
console.log(add(2, 3)); // 5

// Default import (any name)
import Calculator from './math.js';

// Import everything
import * as Math from './math.js';
Math.add(1, 2); // 3
```

■ Quick Reference Cheat Sheet

Concept	Syntax / Example
Variable	<code>let x = 5; const Y = 10;</code>
Arrow function	<code>const fn = (a,b) => a + b;</code>
Template literal	<code>`Hello \${name}`</code>
Destructuring	<code>const {a, b} = obj;</code>
Spread	<code>[...arr1, ...arr2]</code>
Optional chain	<code>obj?.prop?.sub</code>
Nullish coalesc.	<code>val ?? "default"</code>
Array map	<code>arr.map(x => x*2)</code>
Array filter	<code>arr.filter(x => x>0)</code>
Array reduce	<code>arr.reduce((acc,x)=>acc+x, 0)</code>
Async/Await	<code>const data = await fetch(url);</code>
Try/Catch	<code>try { } catch(e) { }</code>
Class	<code>class Dog extends Animal { }</code>
Module export	<code>export const fn = () => { };</code>
Module import	<code>import { fn } from './mod.js';</code>

■ ***You now have a solid foundation in JavaScript! Practice by building small projects: a to-do list, a calculator, a weather app. The best way to learn JS is to write it every day.***